



Analyzing Grok 4.5 Behaviour on Long Horizon and Enterprise Coding Tasks

Independent evaluation report prepared by Specific Labs

Executive finding

Across three production-derived long-horizon tasks, Grok 4.5 solved **0/24** verifier-valid attempts while Claude Opus 5 solved **19/24**. The clearest trace comparison comes from the top-up and S3 tasks, where Grok solved **0/16** attempts and Opus solved **12/16**. Grok often built most of the feature, but missed one or two connections needed to carry a rule or generated value through the full workflow. Across eight bug-fixing tasks, Grok solved **21/80** attempts and Opus solved **55/80**.

Contents

Setup	2
Headline result	2
Enterprise long-horizon results	2
Bug-injection debugging analysis	4
Native-table migration difficulty control	6
What the Traces Show	9
Nature of the source codebases	11
Conclusion	12
Appendix	12

Setup

This report keeps two evaluation tracks and one difficulty control separate. The bug-injection track evaluates eight production-derived tasks with ten valid OpenCode attempts per task for both Grok 4.5 and Claude Opus 5; the enterprise track evaluates three authentic historical features or migrations with eight valid attempts per model. Only runs matching the required route, snapshot, checksum, single-agent policy, and complete hidden-verifier output enter the denominators. The native-table migration remains a separate three-attempt control because Opus 5, Grok 4.5, and GPT-5.6 Sol all scored zero, so it cannot show a weakness that is specific to Grok.

Headline result

Across the full three-task scorecard, Grok 4.5 solved **0/24** attempts and Claude Opus 5 solved **19/24**. The findings in this report focus on the top-up and S3 tasks because both state the expected behavior clearly and check what the finished system does. On those two tasks, Grok is **0/16** and Opus is **12/16**. The billing score remains in the table for completeness, but it is not used to explain Grok's behavior because one check asks for a field in a location that the prompt does not require. The bug-fixing scores are also reported as outcomes, not used to make broader claims about Grok, because some tasks hid the exact boundary being tested while difficulty was calibrated.

Enterprise long-horizon tasks

The headline table uses the unbiased estimator $1 - C(n-c, k) / C(n, k)$. It reports the unweighted macro mean across the three task cells:

Task	Required checks	Model	Solves (c/n)	pass@1	pass@3	pass@8
Macro mean	29 total	Grok 4.5	0/24	0.0000	0.0000	0.0000
Macro mean	29 total	Claude Opus 5	19/24	0.7917	0.9940	1.0000

The solves column sums the three task cells. pass@8 is the task-coverage criterion: 1 when a model solves a task at least once across its eight attempts and 0 otherwise.

Bug-injection debugging tasks

The headline uses the same schema and four-decimal precision as the enterprise table. Each model has ten valid attempts on each of eight tasks:

Task	Required checks	Model	Solves (c/n)	pass@1	pass@3	pass@10
Macro mean	152 total	Grok 4.5	21/80	0.2625	0.3833	0.5000
Macro mean	152 total	Claude Opus 5	55/80	0.6875	0.8344	0.8750

The solves column sums the eight task cells. pass@10 is the task-coverage measure because each cell has ten attempts.

Enterprise long-horizon results

The three tasks come from real features or migrations and touch several parts of a working system:

Task	Reference change	Source history	Work covered
Billing schedule	23 files / 343 LOC	56 files over 4 days	customer enrollment, invoice periods, ledger, queues, empty usage
Top-up lifecycle	28 files / 1,450 LOC	21 commits over 6 days	DTO/entity/persistence, hourly scheduling, wallet credit, invoice and overdraft ordering

Task	Reference change	Source history	Work covered
S3 measurement	17 files / 1,809 LOC	four PRs over 3 days	nested configuration, IAM trust/policy, persistence, connector routing, mirrored DLQ writes

The S3 line count includes a dependency lockfile, so task difficulty is judged from the parts of the system that must work together, not from line count alone. A run passes only when every required behavior and regression check passes.

Pass@k results

Using the unbiased estimator $1 - C(n-c, k) / C(n, k)$, each task/model cell has eight valid attempts. The coverage column is therefore pass@8:

Task	Required checks	Model	Solves (c/n)	pass@1	pass@3	pass@8
Customer billing-schedule migration	8	Grok 4.5	0/8	0.0000	0.0000	0.0000
Customer billing-schedule migration	8	Claude Opus 5	7/8	0.8750	1.0000	1.0000
Top-up billing lifecycle	11	Grok 4.5	0/8	0.0000	0.0000	0.0000
Top-up billing lifecycle	11	Claude Opus 5	7/8	0.8750	1.0000	1.0000
S3 datastore measurement	10	Grok 4.5	0/8	0.0000	0.0000	0.0000
S3 datastore measurement	10	Claude Opus 5	5/8	0.6250	0.9821	1.0000
Macro mean	29 total	Grok 4.5	0/24	0.0000	0.0000	0.0000
Macro mean	29 total	Claude Opus 5	19/24	0.7917	0.9940	1.0000

The summary solve values are sums across task cells; the pass@k summary is the unweighted macro mean of the three task-level estimators, not a pooled 24-attempt estimator. pass@8 is 1 for any task cell with at least one solve and 0 for a zero-solve cell.

Measured effort

Agent wall time excludes sandbox setup and grading. Trial wall time includes those phases and remote scheduling/provider latency.

Model	Task	Turns, mean	Tool calls, mean	Agent time, median (range)	Trial time, range
Grok	Billing	17.3	64.4	2.9m (2.8–3.7m)	3.3m–25.3m
Opus	Billing	62.3	67.1	10.5m (7.9–12.2m)	8.7m–22.8m
Grok	Top-up	54.6	161.4	13.6m (10.6–19.2m)	12.0m–58.6m
Opus	Top-up	149.9	149.6	27.5m (21.6–40.2m)	22.2m–59.7m
Grok	S3	22.1	74.0	5.7m (5.2–18.4m)	5.8m–18.9m
Opus	S3	109.0	111.5	25.1m (19.7–36.3m)	20.3m–37.6m

Across the 48 raw enterprise attempts, the agents produced 3,321 model turns and 5,024 tool calls. Top-up produces the deepest measured trajectories. The billing traces remain part of the score matrix, but they are not used in the behavioral findings below because one remaining check conflicts with the prompt.

Where Grok came closest

Grok did not fully solve an enterprise task, but it completed much of the work on top-up billing and S3 measurement. The table shows the best Grok run on each task and the final pieces that were still missing:

Task	Best Grok run	Opus result	What worked	What was still missing
Top-up lifecycle	9/11	7/8 solve	refill amount, credit storage, hourly balance check, and invoice creation	apply the same validation during create and update; keep one scheduler per offering
S3 measurement	8/10	5/8 solve	IAM setup, saved configuration, ingestion, and dead-letter handling	return the generated access details and save the same values everywhere

These runs show that Grok can find the right files and implement substantial parts of a feature. The remaining errors appear when the same rule must hold in several entry points, or when a value must move from setup through storage to the final response. The detailed examples appear in What the Traces Show.

Fairness and validity

- Untouched bases score 0 and authorized solvability oracles score 1 for all three tasks.
- Every selected attempt matches the exact model route, OpenCode version, Daytona snapshot, single-agent policy, frozen checksum, and complete verifier output.
- An older billing check depended on Nest constructor order instead of behavior. It was replaced, fresh controls passed, and the older runs are not included in the tables.
- Hidden AWS and database assertions run against offline mocks; no external operation leaves the verifier process.
- Trajectories are credential-redacted and the published artifact manifest records SHA-256 hashes.

The complete evidence is available in [long-horizon-enterprise-results.json](#), [enterprise-long-horizon-trials/](#), and [long-horizon-enterprise-artifacts-manifest.json](#).

Bug-injection debugging analysis

Bug-task pass@k results

Each model/task cell has ten verifier-valid attempts. Using the unbiased estimator $1 - C(n-c, k) / C(n, k)$, the task-level comparison is:

Task	Required checks	Model	Solves (c/n)	pass@1	pass@3	pass@10
credit-normalize	19	Grok 4.5	8/10	0.8000	1.0000	1.0000
credit-normalize	19	Claude Opus 5	8/10	0.8000	1.0000	1.0000
doc-extractors	19	Grok 4.5	0/10	0.0000	0.0000	0.0000
doc-extractors	19	Claude Opus 5	6/10	0.6000	0.9667	1.0000
financial-tools	23	Grok 4.5	0/10	0.0000	0.0000	0.0000
financial-tools	23	Claude Opus 5	3/10	0.3000	0.7083	1.0000
phone-invites	17	Grok 4.5	9/10	0.9000	1.0000	1.0000
phone-invites	17	Claude Opus 5	9/10	0.9000	1.0000	1.0000
FIU	19	Grok 4.5	2/10	0.2000	0.5333	1.0000
FIU	19	Claude Opus 5	10/10	1.0000	1.0000	1.0000
txenrich	17	Grok 4.5	2/10	0.2000	0.5333	1.0000
txenrich	17	Claude Opus 5	9/10	0.9000	1.0000	1.0000
txenrich3	19	Grok 4.5	0/10	0.0000	0.0000	0.0000
txenrich3	19	Claude Opus 5	10/10	1.0000	1.0000	1.0000
txenrich4	19	Grok 4.5	0/10	0.0000	0.0000	0.0000

Task	Required checks	Model	Solves (c/n)	pass@1	pass@3	pass@10
txenrich4	19	Claude Opus 5	0/10	0.0000	0.0000	0.0000
Macro mean	152 total	Grok 4.5	21/80	0.2625	0.3833	0.5000
Macro mean	152 total	Claude Opus 5	55/80	0.6875	0.8344	0.8750

The summary solve values are sums across task cells; pass@k is the unweighted macro mean of the eight task-level estimators. pass@10 is the task-coverage measure because each cell has ten attempts. The comparison therefore shows both a repeatability gap, 21/80 versus 55/80, and a breadth gap, four of eight tasks with a Grok solve versus seven of eight with an Opus solve.

Bug-task measured effort

A model turn is an agent-sourced OpenCode trajectory step and records one LLM call. Full trial wall time is Harbor started_at to finished_at, including environment setup, agent setup, execution, and verification.

Model	Valid trials	Model turns	Tool calls	Mean trial wall time	Median	p90	Range
Grok 4.5	80	967	2,521	4m 27.9s	3m 52.0s	8m 33.5s	1m 28.5s-15m 14.5s
Claude Opus 5	80	3,061	3,316	12m 46.1s	10m 14.5s	23m 28.3s	5m 04.3s-30m 28.0s

Model	Task	Turns, mean	Tool calls, mean	Trial wall time, mean	Trial wall time, range
Grok	credit-normalize	10.9	22.3	3.12m	2.48-3.95m
Opus	credit-normalize	33.0	37.5	7.48m	5.18-10.71m
Grok	doc-extractors	9.1	19.1	1.89m	1.59-2.39m
Opus	doc-extractors	25.9	30.9	9.73m	5.65-23.47m
Grok	financial-tools	8.2	21.9	1.70m	1.48-1.85m
Opus	financial-tools	43.9	47.4	10.30m	5.07-17.00m
Grok	phone-invites	8.0	15.8	2.09m	1.80-2.32m
Opus	phone-invites	17.1	18.6	8.13m	7.08-12.71m
Grok	FIU	15.4	46.8	5.44m	4.14-7.49m
Opus	FIU	36.2	42.9	15.26m	7.44-30.47m
Grok	txenrich	14.2	36.4	6.77m	5.51-10.20m
Opus	txenrich	41.7	43.2	13.23m	7.03-22.97m
Grok	txenrich3	14.3	43.5	4.89m	3.78-5.82m
Opus	txenrich3	54.5	55.8	16.90m	9.72-27.17m
Grok	txenrich4	16.6	46.3	9.81m	7.53-15.24m
Opus	txenrich4	53.8	55.3	21.13m	10.24-30.21m

Grok's 80 valid trials ran in a global 12-sandbox pool. The first valid start to last valid finish was **1h 08m 28.8s**, with peak observed concurrency 12. The Opus attempts combine the original exact-route OpenRouter screen with later exact Bedrock-route attempts, so they do not form one comparable single-wave envelope. Independently running trial durations are not summed.

The Grok phase timestamps put the mean trial at 2.2s of environment setup, 20.5s of agent setup, 234.6s of agent execution, and 6.6s of verification, with the small remainder in handoffs. Solved Grok attempts were shorter than unsolved attempts on average, 10.6 versus 12.6 model turns and 3.07m versus 4.96m, but task difficulty confounds that comparison. The same Grok trajectories report **46,841,681 input tokens**, including **39,586,944 cached tokens**, and **467,658 output tokens**.

Where Grok solved the bug-fixing tasks

Grok solved 21 of the 80 bug-fixing runs, but most of those wins came from two tasks:

Task	Grok c/n	Opus c/n	What the task required
credit-normalize	8/10	8/10	fix named text helpers and check exact input examples
phone-invites	9/10	9/10	fix one phone-prefix path and preserve fallback order
FIU	2/10	10/10	find and fix several small related helpers in one run
txenrich	2/10	9/10	change one exact rule without widening nearby bank rules
doc-extractors	0/10	6/10	allow the new valid case while still rejecting the closest invalid case
financial-tools	0/10	3/10	carry one delinquency rule into the final code
txenrich3	0/10	10/10	make the fix in the bank-specific code that handles the failing row
txenrich4	0/10	0/10	make all five parser cases work without breaking old behavior

Grok did well when the problem pointed to one named function. For example, [phone attempt 1](#) correctly changed an international prefix from 00 to + and kept the first configured fallback:

```
if value.startswith("00"):
    value = "+" + value[2:]

if possible_matches:
    return possible_matches[0]
```

That run passed all 17 checks. Grok had more difficulty when several similar rules appeared in different files or when a fix also had to preserve the nearest invalid case. Because some prompts hid those exact details while difficulty was calibrated, this report treats the scores as outcomes rather than using them to explain a Grok-specific weakness.

How to interpret the bug-task results

The scores show which patches passed, but they do not provide a clean answer for why one model failed. They are therefore not used in the training findings later in the report.

Native-table migration difficulty control

This task is reported separately because every model failed all three attempts. It shows a hard problem shared by Grok, Opus, and GPT-5.6 Sol, not a weakness specific to Grok. Tool use ranged from **80 to 169** calls, with a median of **116**. All scored runs used the same OpenCode version, Daytona snapshot, and model routes. Setup failures that ended before the model worked are not counted.

pass@k uses $1 - C(n-c, k) / C(n, k)$. Every model has three valid attempts on the six-check task:

Task	Required checks	Model	Solves (c/n)	pass@1	pass@3	Task coverage (pass@3)
Native-table migration	6	Grok 4.5	0/3	0.0000	0.0000	0.0000
Native-table migration	6	Claude Opus 5	0/3	0.0000	0.0000	0.0000
Native-table migration	6	GPT-5.6 Sol	0/3	0.0000	0.0000	0.0000

Here pass@3 is also task coverage because n=3.

Native-table measured effort

Model	Valid trials	Model turns	Tool calls, median (range)	Trial wall time, mean / median (range)
Grok 4.5	3	110	116 (108–134)	11m 41.9s / 11m 09.2s (10m 11.9s–13m 44.7s)
Claude Opus 5	3	437	168 (148–169)	56m 52.7s / 50m 07.4s (41m 46.6s–78m 44.2s)
GPT-5.6 Sol	3	113	90 (80–90)	8m 24.9s / 8m 41.1s (7m 35.9s–8m 57.8s)

Per-attempt results

Model	Attempt	Reward	f2p	p2p	Model turns	Tool calls	Full trial wall time	Input tokens (cached)	Output tokens
Opus 5	1	0	0/4	1/2	150	169	50m 07.4s	34.89M (34.89M)	143.9k
Opus 5	2	0	1/4	1/2	154	168	78m 44.2s	38.24M (38.24M)	150.7k
Opus 5	3	0	0/4	1/2	133	148	41m 46.6s	26.20M (26.20M)	125.0k
Grok 4.5	1	0	1/4	2/2	39	134	11m 09.2s	2.80M (2.67M)	32.7k
Grok 4.5	2	0	1/4	1/2	36	116	13m 44.7s†	2.72M (2.52M)	31.4k
Grok 4.5	3	0	1/4	2/2	35	108	10m 11.9s	2.71M (2.50M)	28.2k
GPT-5.6 Sol	1	0	1/4	1/2	37	80	7m 35.9s	2.24M (2.24M)	13.4k
GPT-5.6 Sol	2	0	1/4	1/2	40	90	8m 41.1s	2.73M (2.73M)	14.1k
GPT-5.6 Sol	3	0	1/4	1/2	36	90	8m 57.8s	2.73M (2.73M)	15.1k

The comparison used **660 model turns**, **1,103 tool calls**, **115.26M input tokens** (114.72M cached), and **554.3k output tokens**. Independently running trial durations are not summed.

† Eight durations are full Harbor started_at→finished_at wall time, including environment setup, agent setup, agent execution, and verification. In one Grok trial the agent command exited successfully but Harbor stalled while collecting the finished Daytona sandbox. Its 13m 44.7s value is Harbor start to the recovered agent-completion timestamp, conservatively excluding the post-agent collection stall; duration_basis records that distinction.

An earlier checker depended on exact dependency setter names, so a valid refactor could make the checker itself fail to compile. The final checker finds compatible dependencies by type instead. The untouched code scores 0, the historical solution scores 1, and a different valid wiring also scores 1. The Opus candidates were rerun on this final checker and still scored 0; Grok and Sol were run only after it was fixed.

What the native-table task checks

The hidden tests check behavior rather than filenames, class names, or similarity to the historical solution:

Requirement	What a passing result must show
Read grid, box, and text-aligned tables	Three sample PDFs return the expected non-empty cells
Choose the right extraction method	Seven bank and file-format examples select a working method
Skip the remote service after local success	A successful local result reaches the application without calling the remote processor
Keep the existing fallback	An unknown bank still reaches the remote processor and mapper
Report which path was used	Local, ML, and fallback status appears in the API response and logs
Preserve old date behavior	Existing date parsing and blank-input behavior still pass

The untouched code scores 0. The historical solution and a different valid implementation both score 1, which shows that the checker does not require one exact code structure. Raw control outputs remain under long-horizon-controls/.

What the native-table traces show

The nine runs failed in different ways. Every model produced plausible code that compiled, but none proved the complete workflow from reading a PDF to returning accepted rows while preserving the remote fallback.

Model	What it built	Best f2p	Best p2p	Why the workflow still failed
Claude Opus 5	Separate readers, geometry helpers, extraction methods, and validation	1/4	1/2	The application could not construct the final service, and sample PDFs returned empty results
Grok 4.5	A broad table-reading approach followed mainly by package-build checks	1/4	2/2	The application never accepted a local table as usable output
GPT-5.6 Sol	One general extractor presented as three named methods	1/4	1/2	The three methods behaved too similarly, and the unknown-format fallback broke

Across all nine runs, no sample PDF produced an accepted local table. Grok runs 1 and 3 preserved both old behaviors, but still failed local extraction, method selection, and remote-skip checks. The examples below show why compilation was not enough.

Grok: the code compiled, but no PDF produced a usable table

Grok attempt 1 added separate geometry-based builders and hybrid routing. It ended by reporting a successful offline package build. The builder, however, initialized every new table as non-transactional and depended on later analysis to change that state:

```
table.setBankTransactions(false);
```

The checker later received empty cells from every sample PDF. Near the end, Grok also narrowed one helper to a single hard-coded bank:

```
bankStatementV0.setBank("HDFC");
documentExtractionRequest.setBankName("HDFC");
```

It reran the package build, but did not run the sample PDFs through the finished service. The code had many of the right pieces, but no test showed that a real PDF became a non-empty table and skipped the remote service.

GPT-5.6 Sol: three method names still used nearly the same logic

GPT-5.6 Sol attempt 1 introduced three policy names, but routed them through one main table-building algorithm. The clearest policy-specific branch only filtered rows for the text-aligned case:

```
if (policy == NativeTablePolicy.TEXT_ALIGNED_ROW_SELECTED
    && !foundTransaction && !dateRow) {
    continue;
}
```

The other two methods did not get their own layout-specific behavior. The model's tests checked method names and broad table shape; the hidden tests used real sample PDFs. All three Sol runs reported the chosen path but returned empty local results and broke the unknown-format fallback.

Opus 5: a large refactor broke the application's entry point

Opus attempt 2 was the closest Opus run. It added page readers, word extraction, geometry, several table readers, and validation. The application still could not construct the refactored service:

```
AzureDocumentExtractionServiceImpl has no dependency slot for AnalyzeDocuments
```

The final checker already accepted dependencies by type, so this was not an exact-name issue. The refactor no longer exposed a service path the application could use. Another Opus run reached the sample PDFs but still returned empty cells. More code did not replace one end-to-end check through the application's real entry point.

What this control shows

All three models could explore the repository, propose a reasonable design, and compile a large patch. None proved all six behaviors on the sample PDFs:

1. A supported PDF produces the exact non-empty structured rows.
2. The selected policy changes how extraction works, not only its label.
3. A successful native result is accepted downstream and skips the remote call.
4. An unsupported format still calls the remote extractor.
5. The chosen path appears in the API response and persisted diagnostics.
6. Legacy date parsing remains unchanged.

The most useful next step is to make one PDF format work through all six checks, then add the other formats without breaking that path. The same task can provide feedback after each step without adding a new environment.

What the Traces Show

Main finding. On the top-up and S3 tasks, Grok usually found the right files and built much of the requested feature. It failed when one rule or value had to survive every step of the workflow. Opus completed that handoff more reliably, solving **12/16** attempts while Grok solved **0/16**.

We compared the final code, tests, commands, and checker results from all 16 Grok runs on these two tasks with the matching Opus runs. The model's comments are used only to compare what it claimed with what the finished code did. Billing remains in the score table, but not in these findings, because one check expects a field placement that the prompt does not ask for.

1. Grok applied the top-up rules in some places, but not all

Result. Grok's best run passed 9/11 checks. The matching Opus run passed 11/11. Grok still allowed top-up fields in an invalid billing mode and did not create one stable hourly job for each offering.

The task says that top-up fields are valid only for top-up billing, subscriptions cannot use them, and each offering should own one hourly job. Grok added a rule to one optional request field:

```
@Validate(TopUpBillingCycleRule)
@IsOptional()
public billingCycle?: ValidBillingCycles;
```

That rule did not cover every way an offering could be created or updated. Its final message still said that invalid top-up fields were rejected and the hourly job was offering-level. The checker found that both statements were false. This was not a one-off miss: the hourly-job check failed in 8/8 Grok runs, rejection on other billing modes failed in 7/8, and subscription rejection failed in 5/8.

Opus put the same check in the shared create and update paths, and based the job identifier on the offering:

```
OfferingService.validateTopUpFields(createOfferingDTO);
OfferingService.validateTopUpFields(mergedUpdateState);

schedulerID: Offering.getTopUpSchedulerID(this.offeringId),
scheduleParameters: { businessID: this.businessID,
                      offeringId: this.offeringId },
```

The important difference is not the exact helper name. Opus made the rule run wherever the data entered the system, then checked the scheduled path as well.

What to train. For every rule written as “when,” “only if,” or “otherwise,” list each allowed and rejected case. Then test that list through create, update, and scheduled execution before marking the requirement complete.

2. Grok created the S3 values, but lost them before they could be used

Result. Grok attempt 5 passed 5/10 checks. The matching Opus run passed 10/10. Every Grok run failed to return the generated access details and failed to carry them through creation and storage. Six of eight also failed the bad record path.

The task says that the generated IAM role, external ID, ingestion location, dead-letter location, and region must be returned and saved. In Grok attempt 5, the helper filled in the local object but ended without returning it:

```
dbAccessInformation.iamRoleArn = createRoleResponse.Role?.Arn;
dbAccessInformation.externalId = externalId;
dbAccessInformation.ingestion = ingestion;
dbAccessInformation.dlq = dlq;
// method ends; caller receives undefined
```

The caller therefore received `undefined`. The values existed briefly inside the helper, but the rest of the system could not save or use them. Opus returned the configured object, saved it, and then returned the created entity:

```
const configured = await DatastoreAccessInformation.setupAccess(entity);
entity.measurementConfiguration = configured;
const point = MeasurementConfigEntity.transformer(entity, influx);
await loadPoints(configBucket, tenant, point);
return entity;
```

What to train. Follow each generated value through four questions: Was it created? Was it returned? Was it saved? Did the final service return the same value? Run this check for both the normal path and the bad-record path.

3. Grok tested pieces of the feature, then reported the whole job complete

In the top-up run, Grok’s final message said validation and hourly-job identity were complete. Its tests only showed that a service could be constructed; they did not create invalid offerings or inspect the scheduled job. In the S3 run, Grok said setup, storage, and dead-letter delivery worked after running tests that never checked the object returned by the setup helper.

Across these 16 runs, Grok ran a build or test after its final source change in 9 cases; Opus did so in all 16. Grok reviewed its final repository changes in 2 cases; Opus did so in 14. These habits do not cause success by themselves, but the paired examples show what the missing final check would have caught.

What to train. Before reporting completion, run one test through the same entry point a real caller uses. Match every sentence in the final summary to a passing test or an inspected final object. If either is missing, report the open item instead of calling the task complete.

Taken together, the traces show a narrow and practical weakness. Grok can build the individual pieces, but it is less reliable at connecting those pieces across the whole workflow and proving that the final result matches the request.

Nature of the source codebases

The evaluation covers five authorized private production systems. After excluding dependencies, generated and build output, and Git metadata, the agent-visible snapshots contain **1,789 code files** and **269,074 code LOC** across Python, TypeScript, JavaScript, and Java. They also expose **149 test files** and **135 PDF fixtures**. Across the two evaluation tracks and the separate difficulty control, these systems support **12 task instances** and **187 required checks**.

Source system	Language mix by code LOC	Agent-visible snapshot	Evaluation scope	Tasks represented
Financial workflow backend	54.0% Python 46.0% TypeScript	521 code files / 133,848 LOC 33 test files / 3,686 test LOC	4 bug tasks 20 planted defects, 19 reward-gated 78 required checks	credit-normalize doc-extractors financial-tools phone-invites FIU
Financial integration service	100% Java	264 code files / 16,375 LOC 17 test files / 2,823 test LOC	1 bug task 5 planted defects 19 required checks	
Transaction-enrichment service	100% Python	52 code files / 11,179 LOC no candidate-visible test files	3 bug tasks 15 planted defects 55 required checks	txenrich txenrich3 txenrich4
Billing and measurement platform	99.5% TypeScript 0.5% JavaScript	539 code files / 66,319 LOC 99 test files / 20,475 test LOC	3 long-horizon tasks 68 oracle files / 3,602 changed LOC 29 required checks	Customer billing-schedule migration Top-up billing lifecycle S3 datastore measurement
Document-processing platform	100% Java	413 code files / 41,353 LOC 135 PDF fixtures	1 difficulty control 62 commits / 70 files / approximately 13,000 added LOC 6 required checks	Native-table migration difficulty control

Code-file and LOC counts include candidate-visible tests; the test counts are subsets shown separately. Dependency trees, compiled output, generated output, and Git metadata are excluded. The transaction-enrichment snapshot intentionally has no candidate-visible tests, but its three tasks are independently graded by 55 hidden checks.

The eight bug-injection tasks alter **40 narrow boundaries**, of which **39 are reward-gated**, while preserving the surrounding systems. The three enterprise long-horizon tasks start from pre-feature production revisions and use historical changes spanning **68 oracle files** and **3,602 changed LOC** only as solvability references. The separate native-table control condenses **62 production commits**, **70 files**, and approximately **13,000 added LOC** into one six-check task. Independently written hidden tests grade observable behavior rather than source similarity.

Company names, ticket identifiers, commit messages, credentials, customer records, and unrelated configuration are removed, while the architecture and task-relevant dependencies remain intact. The evaluation therefore rests on measured code volume, verifier scope, behavior, controls, and traces rather than the reputation of any source company.

Conclusion

On the two enterprise tasks with clear end-to-end requirements, Grok often implemented most of the requested feature. The remaining failures came from rules that were applied in one path but not another, values that were created but not returned or saved, and tests that checked individual pieces instead of the complete workflow.

What Grok can improve	Evidence	What to train
Apply each rule everywhere it matters	Top-up billing lifecycle	List each allowed and rejected case, then test it during create, update, and scheduled execution.
Carry generated values through the full workflow	S3 datastore measurement	Check that each value is created, returned, saved, and returned again by the service, including the bad-record path.
Prove the complete behavior before stopping	Top-up billing lifecycle S3 datastore measurement	Run one test through the real entry point and tie every completion claim to a passing check or inspected final object.

The practical target is straightforward: keep every requirement active until the final code and an end-to-end test show that it works across the complete workflow.

Appendix

Long-horizon definition and evaluation bar

There is no standard 70-turn or raw-token threshold for a long-horizon task. The term is used along at least two independent axes:

- **DeepSWE** treats long horizon as a structural property: short, natural prompts whose solutions require substantial codebase exploration and large, multi-file implementations. It reports tokens and trial time as efficiency measurements, not as the definition.
- **SWE-Bench Pro** uses an estimated professional engineering horizon of hours to days together with multi-file, substantial code modifications.
- **SWE-EVO** tests release-level evolution rather than a single issue; its reference patch edits 20.9 files and 610.5 lines on average, with an average 874-test evaluation surface.
- **METR** defines a model time horizon using the human-expert completion time at which the model has a target success probability. It explicitly does not mean agent elapsed time or action count.

That variation is important. Recent non-coding suites labeled long horizon range from [over 20 tool calls per task in WildClawBench](#) to [318 on average in OSWorld 2.0](#), while [TRIP-Bench](#) reports trajectories reaching 150+ tool calls and 200k+ context tokens. Turn, tool, and token counts are harness- and domain-dependent diagnostics, not portable cutoffs.

The primary bar for this package is therefore repository-scale structure. The three capability-gap tasks cross persistence, validation, scheduling, billing, queueing, IAM, connector, and failure-recovery boundaries. Their packaged oracles span 17–28 files, and their source evidence covers multi-day production changes. The separate native-table task condenses 62 production commits across 70 files and several dependent subsystems into a six-requirement prompt.

Turn, tool, token, and wall-time measurements describe the resulting agent behavior; they are not substituted for structural scope or verifier outcome. No controlled human completion-time baseline was collected, so the tasks are not assigned METR-style hour values. Commit count and calendar span establish provenance and dependency depth, not human labor time.

The capability-gap cohort uses eight valid attempts per model and task. A task is learnable when Grok solves

one to six attempts, or when Grok solves zero and a comparable model completes the task. The native-table control instead asks whether Opus 5 fails at least half of its independent valid attempts. In both cohorts, only trials whose hidden verifier returns complete per-test verdicts enter the denominator; provider, verifier, network, and operator failures without verdicts are excluded.